



Git Versionskontrolle

Daniel Senften

Version 1.1, 2025-10-14

Inhaltsverzeichnis

1. Einführung	1
1.1. Was ist Git?	1
1.2. Warum Versionskontrolle verwenden?	1
2. Installation und Einrichtung	2
2.1. Systemvoraussetzungen	2
2.2. Plattformspezifische Installation	2
2.3. Erste Schritte nach der Installation	3
3. Grundlagen der Versionskontrolle	4
3.1. Repository initialisieren	4
3.2. Der typische Workflow	4
3.3. Verzeichnisstruktur verstehen	5
4. Git Befehle im Überblick	7
4.1. Repository erstellen und klonen	7
4.2. Grundlegende Workflow-Befehle	7
4.3. Branching und Merging	7
4.4. Fortgeschrittene Befehle	7
5. Git Workflows	8
5.1. Zentrale Workflows	8
5.2. Gitflow im Detail	8
6. Arbeiten mit Remote-Repositories	10
6.1. Grundlegende Operationen	10
6.2. Klonen und Forken	10
7. Best Practices	11
7.1. Sinnvolle Commits	11
7.2. .gitignore verwenden	11
8. Häufige Probleme und Lösungen	12
8.1. Merge-Konflikte lösen	12
8.2. Änderungen rückgängig machen	12
9. Übungsaufgaben	13
10. Nützliche Ressourcen	14
11. Zusammenfassung	15
11.1. Weiterführende Ressourcen	15
11.2. Zusammenfassung	15

1. Einführung

In der modernen Softwareentwicklung ist die Versionskontrolle nicht mehr wegzudenken. Sie bildet das Fundament für strukturierte, nachvollziehbare und kollaborative Softwareentwicklung.

1.1. Was ist Git?

Git ist ein verteiltes Versionskontrollsystem, das 2005 von Linus Torvalds entwickelt wurde, um die Entwicklung des Linux-Kernels zu verwalten. Es hat sich zum De-facto-Standard in der Softwareentwicklung entwickelt.

Im Gegensatz zu älteren Versionskontrollsystemen ist Git:

- **Dezentral:** Jeder Entwickler hat eine vollständige Kopie des Repositories
- **Schnell:** Optimiert für Performance
- **Branching-fokussiert:** Ermöglicht effizientes paralleles Arbeiten
- **Sicher:** Jeder Commit ist kryptografisch gesichert

1.2. Warum Versionskontrolle verwenden?

- **Änderungsverfolgung:** Dokumentation aller Änderungen am Quellcode
- **Zeitreisen durch Code:** Jeder Stand des Projekts kann wiederhergestellt werden
- **Zusammenarbeit:** Mehrere Entwickler können gleichzeitig an einem Projekt arbeiten
- **Fehleranalyse:** Nachvollziehen, wann und warum Fehler eingeführt wurden
- **Experimentieren:** Neue Funktionen können in separaten Branches entwickelt werden
- **Backup:** Sicherung aller Versionen des Codes

2. Installation und Einrichtung

2.1. Systemvoraussetzungen

- Mindestens 100 MB freier Festplattenspeicher
- Internetverbindung für den Download
- Terminal/Command Line Zugriff

2.2. Plattformspezifische Installation

2.2.1. Windows

1. Lade den Installer von [Git für Windows](#) herunter
2. Führe die .exe-Datei aus und folge den Anweisungen
3. Wähle "Git from the command line and also from 3rd-party software"
4. Nach der Installation öffne Git Bash oder die Eingabeaufforderung

2.2.2. Linux

```
# Debian/Ubuntu
sudo apt update
sudo apt install git

# Fedora
sudo dnf install git

# Arch Linux
sudo pacman -S git
```

2.2.3. macOS

1. Installiere Homebrew falls noch nicht vorhanden:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

- + 2. Installiere Git:

```
brew install git
```

2.3. Erste Schritte nach der Installation

Konfiguriere deine Benutzerdaten:

```
# Setzen des Benutzernamens
git config --global user.name "Max Mustermann"

# Setzen der E-Mail-Adresse
git config --global user.email "max.mustermann@beispiel.ch"

# Standard-Editor auf VS Code setzen
git config --global core.editor "code --wait"

# Nützliche Aliase
[alias]
    co = checkout
    br = branch
    ci = commit
    st = status
    last = log -1 HEAD
    lg = log --graph --pretty=format:'%Cred%h%Creset -%C(yellow)%d%Creset %s
%Cgreen(%cr) %C(bold blue)<%an>%Creset' --abbrev-commit

# Überprüfen der Konfiguration
git config --list
```

3. Grundlagen der Versionskontrolle

3.1. Repository initialisieren

```
# Neues Verzeichnis erstellen
mkdir mein-projekt
cd mein-projekt

# Git-Repository initialisieren
git init

# Status überprüfen
git status
```

3.2. Der typische Workflow

Der typische Workflow in Git besteht aus drei Stufen:

1. **Working Directory** (Arbeitsverzeichnis): Hier werden Änderungen vorgenommen
2. **Staging Area** (Zwischenablage): Vorbereitung der Änderungen für den Commit
3. **Repository**: Finale Speicherung der Versionen

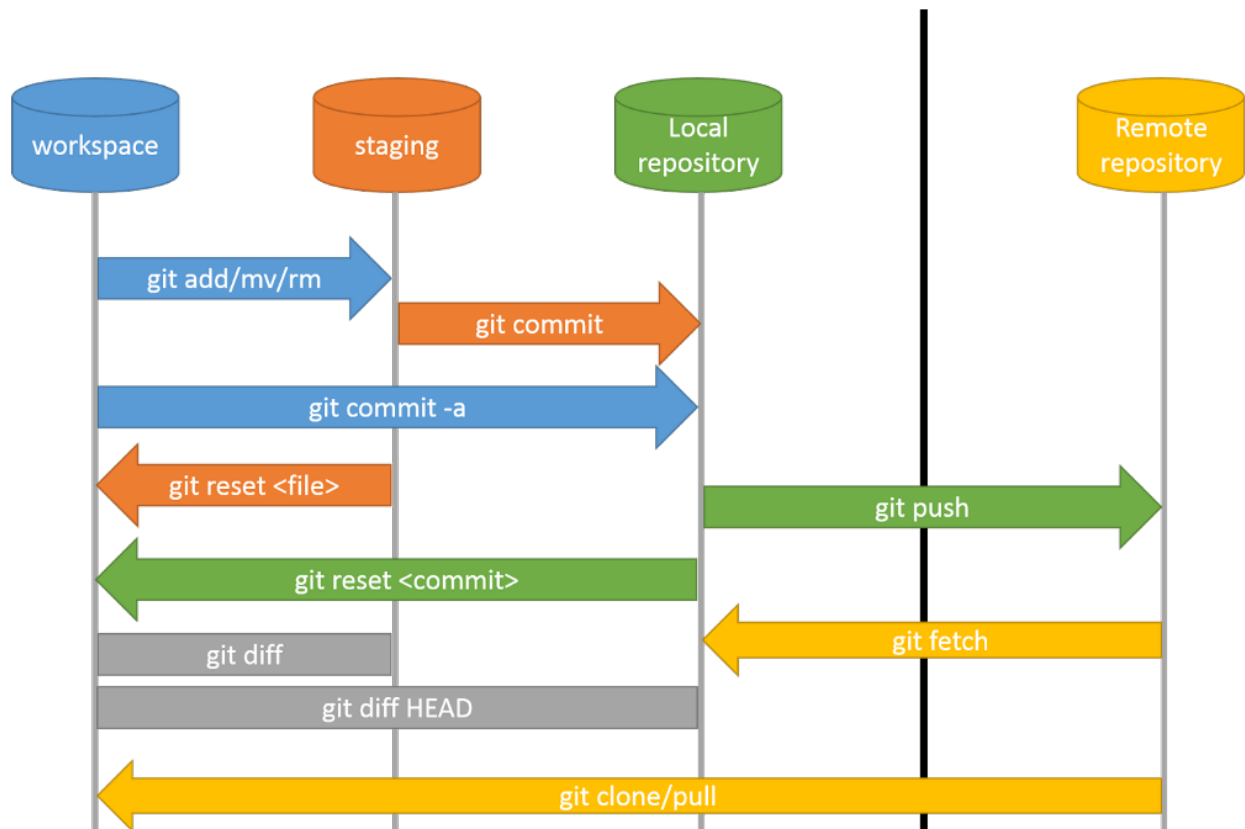


Abbildung 1. Git Grundlegende Befehle

Die Grafik oben zeigt den typischen Git-Workflow und die wichtigsten Befehle, die Sie für die tägliche Arbeit mit Git benötigen.

```
# Änderungen im Arbeitsverzeichnis anzeigen
git status

# Bestimmte Dateien zur Staging-Area hinzufügen
git add README.md index.html

# Alle geänderten Dateien hinzufügen
git add .

# Commit mit Nachricht erstellen
git commit -m "Initiale Version des Projekts"
```

3.3. Verzeichnisstruktur verstehen

Ein typisches Git-Repository enthält:

- **.git/**: Verzeichnis mit allen Metadaten
- **HEAD**: Zeiger auf den aktuellen Branch

- `config`: Repository-Konfiguration
- `objects/`: Enthält alle Repository-Daten
- `refs/`: Zeiger auf Commit-Objekte

4. Git Befehle im Überblick

4.1. Repository erstellen und klonen

Befehl	Beschreibung
<code>git init</code>	Erstellt ein neues Git-Repository im aktuellen Verzeichnis
<code>git clone <url></code>	Klont ein bestehendes Repository

4.2. Grundlegende Workflow-Befehle

Befehl	Beschreibung
<code>git status</code>	Zeigt den aktuellen Status des Arbeitsverzeichnisses
<code>git add <datei></code>	Fügt Änderungen zur Staging-Area hinzu
<code>git commit -m "Nachricht"</code>	Erstellt einen Commit mit den gestagten Änderungen
<code>git push</code>	Überträgt lokale Commits zum Remote-Repository
<code>git pull</code>	Holt Änderungen vom Remote-Repository und führt sie zusammen

4.3. Branching und Merging

Befehl	Beschreibung
<code>git branch</code>	Zeigt alle lokalen Branches an
<code>git branch <name></code>	Erstellt einen neuen Branch
<code>git checkout <branch></code>	Wechselt zu einem anderen Branch
<code>git merge <branch></code>	Führt einen Branch in den aktuellen Branch zusammen
<code>git rebase <branch></code>	Wendet Commits auf einen anderen Basis-Branch an

4.4. Fortgeschrittene Befehle

Befehl	Beschreibung
<code>git stash</code>	Speichert nicht-committete Änderungen temporär
<code>git cherry-pick <commit></code>	Übernimmt spezifische Commits in den aktuellen Branch
<code>git reset</code>	Setzt den aktuellen HEAD auf einen bestimmten Zustand zurück
<code>git reflog</code>	Zeigt eine Chronik aller Git-Aktionen
<code>git bisect</code>	Hilft bei der Fehlersuche durch binäre Suche

5. Git Workflows

5.1. Zentrale Workflows

Workflow	Beschreibung
Centralized Workflow	Ein zentrales Repository, an dem alle Entwickler arbeiten
Feature Branch Workflow	Jede neue Funktion wird in einem separaten Branch entwickelt
Gitflow Workflow	Strengerer Workflow mit definierten Branch-Typen
Forking Workflow	Jeder Entwickler hat sein eigenes öffentliches Repository

5.2. Gitflow im Detail

Der Gitflow-Workflow ist besonders für Projekte mit regelmässigen Releases geeignet:

- **main/master**: Enthält die Produktionsreleases
- **develop**: Hauptentwicklungszweig
- **feature/**: Für neue Funktionen
- **release/**: Für die Vorbereitung neuer Releases
- **hotfix/**: Für dringende Korrekturen am Produktivcode

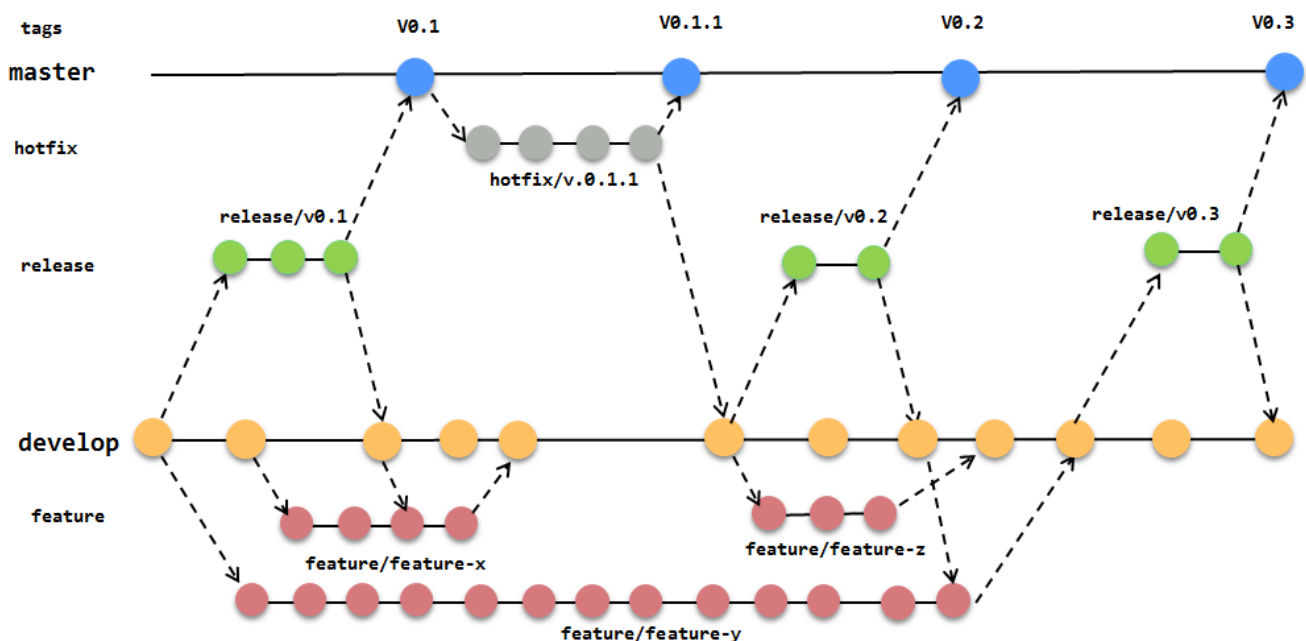


Abbildung 2. Gitflow Workflow Beispiel

Die Grafik veranschaulicht den typischen Gitflow-Workflow mit seinen verschiedenen Branch-Typen und deren Lebenszyklen. Sie zeigt, wie Feature-Branche von **develop** abzweigen, wie Release-Branche für die Vorbereitung neuer Versionen verwendet werden und wie Hotfixes direkt auf den **main**-Branch angewendet werden können.

```
# Beispiel: Neuen Feature-Branch erstellen  
git checkout develop  
git checkout -b feature/neue-funktion
```

6. Arbeiten mit Remote-Repositories

6.1. Grundlegende Operationen

```
# Remote-Repository hinzufügen
git remote add origin https://github.com/benutzername/repository.git

# Überprüfen der Remote-Verbindung
git remote -v

# Lokale Änderungen hochladen
git push -u origin main

# Änderungen vom Remote-Repository holen und mergen
git pull origin main

# Änderungen nur herunterladen
git fetch origin
```

6.2. Klonen und Forken

```
# Repository klonen
git clone https://github.com/benutzername/repository.git

# Alle Branches anzeigen
git branch -a

# Bestimmten Branch auschecken
git checkout -b feature-branch origin/feature-branch
```

7. Best Practices

7.1. Sinnvolle Commits

Gute Commit-Messages sind essentiell für die Nachvollziehbarkeit:

- Verwende eine prägnante Betreffzeile (max. 50 Zeichen)
- Füge bei Bedarf einen ausführlicheren Beschreibungstext hinzu (getrennt durch eine Leerzeile)
- Verwende die imperative Form ("Add feature" statt "Added feature")

Beispiel für eine gute Commit-Nachricht:

```
git commit -m "feat: Implementiere Login-Funktionalität"
```

- Füge Benutzerauthentifizierung hinzu
- Implementiere Session-Handling
- Erstelle grundlegendes Login-Formular"

7.2. .gitignore verwenden

Die **.gitignore**-Datei definiert, welche Dateien nicht versioniert werden sollen:

```
# Kompilierte Dateien
*.class
*.exe
*.dll

# Abhängigkeiten
node_modules/
vendor/

# Umgebungsvariablen
.env

# Betriebssystem-Dateien
.DS_Store
Thumbs.db

# Editor-Dateien
.idea/
.vscode/
*.swp
```

8. Häufige Probleme und Lösungen

8.1. Merge-Konflikte lösen

```
# Status der Konflikte anzeigen
git status

# Konflikt in Datei beheben
# (Datei manuell bearbeiten, Markierungen entfernen)

# Konflikt als gelöst markieren
git add konflikt-datei.txt

# Commit abschliessen
git commit -m "fix: Löse Merge-Konflikt in konflikt-datei.txt"
```

8.2. Änderungen rückgängig machen

```
# Letzten Commit verwerfen, Änderungen behalten
git reset --soft HEAD~1

# Letzten Commit komplett verwerfen
git reset --hard HEAD~1

# Änderungen in einer Datei verwerfen
git checkout -- dateiname.txt

# Nicht getrackte Dateien löschen
git clean -fd
```

9. Übungsaufgaben

1. Initialisieren Sie ein neues Git-Repository
2. Erstellen Sie eine README.md mit Projektbeschreibung
3. Committen Sie die README.md
4. Erstellen Sie einen Feature-Branch
5. Nehmen Sie Änderungen vor und committen Sie diese
6. Wechseln Sie zurück zum main-Branch
7. Mergen Sie den Feature-Branch
8. Erstellen Sie ein Repository auf GitHub
9. Verbinden Sie Ihr lokales Repository mit GitHub
10. Pushen Sie Ihre Änderungen

10. Nützliche Ressourcen

- Offizielle Git-Dokumentation: <https://git-scm.com/doc>
- Pro Git Buch (deutsch): <https://git-scm.com/book/de/v2>
- GitHub Learning Lab: <https://lab.github.com/>
- Git Cheat Sheet: <https://education.github.com/git-cheat-sheet-education.pdf>

11. Zusammenfassung

Git ist ein mächtiges Werkzeug, dessen Beherrschung für jeden Softwareentwickler unerlässlich ist. Dieses Tutorial hat Ihnen die grundlegenden Konzepte und Befehle vermittelt. Durch regelmässige Übung und Anwendung werden Sie schnell in der Lage sein, Git effektiv in Ihren Projekten einzusetzen.

Denken Sie daran:

- Häufig committen
- Klare, beschreibende Commit-Nachrichten schreiben
- Regelmässig pushen und pullen
- Branches für neue Features verwenden
- Konflikte frühzeitig lösen

11.1. Weiterführende Ressourcen

- Official Git Documentation: <https://git-scm.com/doc>
- Pro Git Book: <https://git-scm.com/book/de/v2>
- Git Cheat Sheet: <https://education.github.com/git-cheat-sheet-education.pdf>

11.2. Zusammenfassung

Git ist ein mächtiges Werkzeug, dessen Beherrschung für jeden Softwareentwickler unerlässlich ist. Dieses Tutorial hat Ihnen die grundlegenden Konzepte und Befehle vermittelt. Durch regelmässige Übung und Anwendung werden Sie schnell in der Lage sein, Git effektiv in Ihren Projekten einzusetzen.